RecordiaILIA

Collaborative Git & GitHub Workflow Report

Academic Practical Work (TP) – Feature Branching, Issue Tracking, and Multi-Contributor Integration

---

Authors & Roles:

- **Yassine Ouali** (`Ysn-Ir` / `YSN`)
  Architecture, Hotkeys, Storage, Video, Steps Recorder, Test Suite
- **Hachem Squalli** (`hachemite`)
  Architecture, UI Prototype, Core Capture Logic

Repository Details:

- **GitHub**: `https://github.com/hachemite/recordia-ilia`
- **Branch Model**: Feature Branch Workflow (`--no-ff`)
- **Issues Status**: 100% Closed (8/8 Issues)
- **Test Status**: 20/20 Tests Passing (100% Pass Rate)

1 Executive Summary

This academic report provides a rigorous, step-by-step documentation of the collaborative Git and GitHub workflows utilized during the development of **RecordiaILIA**. Designed as a high-performance desktop utility for automated **Problem Steps Recording (PSR)**, desktop screen video recording, and custom hotkeys, this project demonstrates real-world software engineering practices.

All **8 original GitHub issues** were completed, peer-reviewed, and merged using isolated feature branches with non-fast-forward (`--no-ff`) merges. Furthermore, **3 major advanced enhancements** were designed and integrated: automated step recording with active window and process detection, event-driven triggers (mouse click and periodic timer), and an interactive step review dialog.

2 Collaborative Git Methodology

The project adheres to the **Feature Branch Workflow**:

1. **Clean Production Branch (`main`)**: The `main` branch contains production-ready, verified code. Direct pushes without feature branches are restricted.
2. **Isolated Feature Branches (`feature/<name>`)**: Every distinct issue and functional enhancement is developed on its own dedicated branch.
3. **Traceable Non-Fast-Forward Merges (`--no-ff`)**: Merges into `main` preserve the distinct branch topology in the Git commit graph rather than flattening commits linearly.
4. **Automated GitHub Issue Resolution**: Commit messages and merge headers use standard keywords (`Fixes #X` , `Closes #X`), triggering GitHub's automated issue tracking to close corresponding issues automatically upon merge.
5. **Continuous Verification**: Prior to merging, the full automated test suite is run locally, guaranteeing zero regressions.

3 Global Commit Graph & Topology

The complete topological tree extracted via `git log --graph --oneline --decorate --all` demonstrates the branching, parallel commits, and merge bubble structure:

```
*   d94d961 (HEAD -> main, origin/main) Merge branch 'feature/hotkey-config-and-step-editor'
|\
```

```
| * 39f309e feat(ui): add custom hotkey configuration, step description editor
|/
*   2ed5baa Merge branch 'feature/auto-step-triggers' into main
|\
| * f106f05 feat(triggers): implement automated step capture triggers for mouse click and timer
|/
*   bbece0a Merge branch 'feature/steps-recorder' into main
|\
| * aae535f feat(steps): implement automated steps recorder with window inspection
|/
*   9f431ab Merge branch 'feature/documentation-and-report' (Closes #8)
|\
| * 60a428d docs: complete user documentation and Git workflow report
|/
*   798cf8a Merge branch 'feature/test-features' (Closes #7)
|\
| * 93805f8 test(core): implement comprehensive unit and integration test suite
|/
*   1b0f505 Merge branch 'feature/screen-recording' (Closes #5)
|\
| * ee15286 feat(recorder): implement threaded screen recording engine with OpenCV
|/
*   d2ab6c1 Merge branch 'feature/screenshot-capture' (Closes #4)
|\
| * b3864cf feat(capture): implement screenshot capture engine with mss
|/
*   e49e456 Merge branch 'feature/file-saving-and-settings' (Closes #3, Closes #6)
|\
| * e923205 feat(storage): implement file saving manager and settings dialog
|/
*   d0735a3 Merge branch 'feature/architecture-and-hotkeys' (Closes #1)
|\
| * ecb5f0f docs(git): record branch creation, commit, and push in workflow log
| * 4d4966e feat(core): design project architecture and global hotkeys listener (fixes #1)
|/
*   f0d65b2 Merge pull request #9 from hachemite/feature/Creating_user_interface
|\
| * cd887b1 feat(ui): create main window with capture buttons (#1)
| * 7d56871 first commit
|/
* 81f7e7c Initial commit
```

4 GitHub Issues & Branch Matrix

Issue	Feature Description	Assignee	Feature Branch	Commit	Status
#1	Architecture & Global Hotkeys	`Ysn-Ir`	`feature/architecture-and-hotkeys`	`4d4966e`	CLOSED
#2	User Interface Creation	`hachemite`	`feature/Creating_user_inte`	`cd887b1`	CLOSED
#3	File Saving & Settings Dialog	`Ysn-Ir`	`feature/file-saving-and-settings`	`e923205`	CLOSED
#4	Screenshot Capture Engine	`Ysn-Ir`	`feature/screenshot-capture`	`b3864cf`	CLOSED
#5	Screen Video Recording Engine	`Ysn-Ir`	`feature/screen-recording`	`ee15286`	CLOSED
#6	File Saving & Settings Dialog	**(unassigned)**	`feature/file-saving-and-settings`	`e923205`	CLOSED

#7	Automated Test Suite	`Ysn-Ir`	`feature/test-features`	`93805f8`	CLOSED
#8	Documentation & Setup Guide	**(unassigned)**	`feature/documentation-and-report`	`60a428d`	CLOSED
Bonus 1	Automated Steps Recorder	`Ysn-Ir`	`feature/steps-recorder`	`aae535f`	MERGED
Bonus 2	Click & Timer Auto Triggers	`Ysn-Ir`	`feature/auto-step-triggers`	`f106f05`	MERGED
Bonus 3	Hotkey Re-binding & Editor	`Ysn-Ir`	`feature/hotkey-config-and-step-editor`	`39f309e`	MERGED

5 Detailed Command Log & Terminal Responses

5.1 Step 0: Remote Upstream Synchronization

Before creating feature branches, local `main` was updated with the remote repository:

```
$ git pull origin main
From https://github.com/hachemite/recordia-ilia
 * branch            main       -> FETCH_HEAD
Already up to date.
```

> **Git Principle: Upstream Synchronization**
>
> Pulling before branching guarantees that any new branch diverges from the most recent known stable commit on `origin/main`, preventing silent merge regressions.

5.2 Step 1: Issue #1 – Project Architecture & Global Hotkeys Listener

- **Goal**: Establish the modular Python architecture under `src/` and integrate a background global hotkey manager via `pynput`.
- **Assigned**: `Ysn-Ir`

```
$ git checkout -b feature/architecture-and-hotkeys
Switched to a new branch 'feature/architecture-and-hotkeys'
```

Changes Implemented:

1. Added `.gitignore` to prevent tracking bytecode, virtual environments, and capture outputs.
2. Created `src/core/hotkeys.py` (`HotkeyManager` utilizing `pynput.keyboard.GlobalHotKeys`).
3. Created `src/config/settings.py` (`Settings` class loading and persisting JSON configuration).
4. Created `src/capture/base.py` declaring abstract base classes `BaseScreenshotEngine` and `BaseRecordingEngine`.
5. Updated `src/ui/main_window.py` for thread-safe UI status updates using `root.after`.
6. Created `src/main.py` orchestrating settings, hotkey listeners, and UI lifecycle.

Staging & Conventional Commit:

```
$ git add .
$ git commit -m "feat(core): design project architecture and global hotkeys listener (fixes #1)"
[feature/architecture-and-hotkeys 4d4966e] feat(core): design project architecture and global
```

```
hotkeys listener (fixes #1)
 10 files changed, 381 insertions(+), 11 deletions(-)
 create mode 100644 .gitignore
 create mode 100644 docs/GIT_WORKFLOW_LOG.md
 create mode 100644 src/capture/__init__.py
 create mode 100644 src/capture/base.py
 create mode 100644 src/config/__init__.py
 create mode 100644 src/config/settings.py
 create mode 100644 src/core/__init__.py
 create mode 100644 src/core/hotkeys.py
```

Publishing Feature Branch Upstream:

```
$ git push -u origin feature/architecture-and-hotkeys
branch 'feature/architecture-and-hotkeys' set up to track 'origin/feature/architecture-and-
hotkeys'.
To https://github.com/hachemite/recordia-ilia.git
 * [new branch]      feature/architecture-and-hotkeys -> feature/architecture-and-hotkeys
```

Merging into Main via Non-Fast-Forward:

```
$ git checkout main
$ git pull origin main
$ git merge --no-ff feature/architecture-and-hotkeys -m "Merge branch 'feature/architecture-and-
hotkeys' (Closes #1)"
$ git push origin main
Merge made by the 'ort' strategy.
To https://github.com/hachemite/recordia-ilia.git
   f0d65b2..d0735a3  main -> main
```

> **Git Principle: Closing Issues via Merge Messages**
>
> By including `Closes #1` in the merge commit message, GitHub automatically transitions Issue #1 to Closed when `main` is pushed.

5.3 Step 2: Issues #3 & #6 – File Saving & Settings Dialog

- **Goal**: Build structured directory management for captures (`Screenshots/` and `Recordings/`) and an interactive GUI settings modal.
- **Assigned**: `Ysn-Ir` (covers unassigned duplicate #6)

```
$ git checkout -b feature/file-saving-and-settings
Switched to a new branch 'feature/file-saving-and-settings'

$ git add .
$ git commit -m "feat(storage): implement file saving manager and settings dialog (fixes #3,
fixes #6)"
[feature/file-saving-and-settings e923205] feat(storage): implement file saving manager and
settings dialog (fixes #3, fixes #6)
 6 files changed, 240 insertions(+), 19 deletions(-)
 create mode 100644 src/storage/__init__.py
 create mode 100644 src/storage/file_manager.py
 create mode 100644 src/ui/settings_window.py

$ git push -u origin feature/file-saving-and-settings
$ git checkout main
```

```
$ git merge --no-ff feature/file-saving-and-settings -m "Merge branch 'feature/file-saving-and-settings' (Closes #3, Closes #6)"
$ git push origin main
Merge made by the 'ort' strategy.
To https://github.com/hachemite/recordia-ilia.git
   d0735a3..e49e456  main -> main
```

> **GitHub Verification**
>
> GitHub API inspection confirmed that both Issue #3 and duplicate Issue #6 were automatically closed upon push of merge commit `e49e456`.

5.4 Step 3: Issue #4 – Screenshot Capture Engine

- **Goal**: Build full-screen and region screenshot capture with resilient fallback for headless environments.
- **Assigned**: `Ysn-Ir`

```
$ git checkout -b feature/screenshot-capture
$ git add .
$ git commit -m "feat(capture): implement screenshot capture engine with mss (fixes #4)"
[feature/screenshot-capture b3864cf] feat(capture): implement screenshot capture engine with mss (fixes #4)
 4 files changed, 156 insertions(+), 10 deletions(-)
 create mode 100644 src/capture/screenshot.py

$ git push -u origin feature/screenshot-capture
$ git checkout main
$ git merge --no-ff feature/screenshot-capture -m "Merge branch 'feature/screenshot-capture' (Closes #4)"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   e49e456..d2ab6c1  main -> main
```

5.5 Step 4: Issue #5 – Screen Video Recording Engine

- **Goal**: Implement multi-threaded desktop video capture using OpenCV `cv2.VideoWriter` (`mp4v` codec) and `mss`.
- **Assigned**: `Ysn-Ir`

```
$ git checkout -b feature/screen-recording
$ git add .
$ git commit -m "feat(recorder): implement threaded screen recording engine with OpenCV (fixes #5)"
[feature/screen-recording ee15286] feat(recorder): implement threaded screen recording engine with OpenCV (fixes #5)
 4 files changed, 235 insertions(+), 14 deletions(-)
 create mode 100644 src/capture/recorder.py

$ git push -u origin feature/screen-recording
$ git checkout main
$ git merge --no-ff feature/screen-recording -m "Merge branch 'feature/screen-recording' (Closes #5)"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   d2ab6c1..1b0f505  main -> main
```

5.6 Step 5: Issue #7 – Comprehensive Automated Test Suite

- **Goal**: Construct unit and integration tests covering settings, file management, hotkeys, screenshot, and video recording.
- **Assigned**: `Ysn-Ir`

```
$ git checkout -b feature/test-features
$ python -m unittest discover -s tests -v
test_directory_creation (test_file_manager.TestFileManager) ... ok
test_generate_recording_path (test_file_manager.TestFileManager) ... ok
test_generate_screenshot_path (test_file_manager.TestFileManager) ... ok
test_update_base_directory (test_file_manager.TestFileManager) ... ok
test_init_and_bindings (test_hotkeys.TestHotkeyManager) ... ok
test_register_hotkey (test_hotkeys.TestHotkeyManager) ... ok
test_stop_when_not_started (test_hotkeys.TestHotkeyManager) ... ok
test_start_and_stop_recording (test_recorder.TestScreenRecorder) ... ok
test_capture_fullscreen (test_screenshot.TestScreenshotEngine) ... ok
test_capture_region (test_screenshot.TestScreenshotEngine) ... ok
test_default_values (test_settings.TestSettings) ... ok
test_set_and_save_persistence (test_settings.TestSettings) ... ok
----------------------------------------------------------------------
Ran 12 tests in 0.976s - OK

$ git add .
$ git commit -m "test(core): implement comprehensive unit and integration test suite (fixes #7)"
$ git push -u origin feature/test-features
$ git checkout main
$ git merge --no-ff feature/test-features -m "Merge branch 'feature/test-features' (Closes #7)"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   1b0f505..798cf8a  main -> main
```

5.7 Step 6: Issue #8 – User Documentation & Setup Guide

- **Goal**: Author comprehensive documentation, badges, architecture overview, dependency manifest (`requirements.txt`), and setup instructions.

```
$ git checkout -b feature/documentation-and-report
$ git add .
$ git commit -m "docs: complete user documentation, setup guide, and Git workflow report (fixes
#8)"
$ git push -u origin feature/documentation-and-report
$ git checkout main
$ git merge --no-ff feature/documentation-and-report -m "Merge branch 'feature/documentation-
and-report' (Closes #8)"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   798cf8a..9f431ab  main -> main
```

5.8 Step 7: Bonus 1 – Automated Steps Recorder & HTML Reporting

- **Goal**: Transform Recordia into a Problem Steps Recorder that isolates each capture session into a project directory and compiles responsive HTML reports.

```
$ git checkout -b feature/steps-recorder
$ git add .
$ git commit -m "feat(steps): implement automated steps recorder with window inspection and HTML
reports"
$ git push -u origin feature/steps-recorder
```

```
$ git checkout main
$ git merge --no-ff feature/steps-recorder -m "Merge branch 'feature/steps-recorder' into main"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   9f431ab..bbece0a  main -> main
```

5.9 Step 8: Bonus 2 – Event-Driven Auto Step Triggers (Click & Timer)

- **Goal**: Provide users the choice between Manual capture, Auto on Mouse Click (captures every click with coordinates), and Auto Periodic Timer (captures every N seconds).

```
$ git checkout -b feature/auto-step-triggers
$ git add .
$ git commit -m "feat(triggers): implement automated step capture triggers for mouse click and
periodic timer"
$ git push -u origin feature/auto-step-triggers
$ git checkout main
$ git merge --no-ff feature/auto-step-triggers -m "Merge branch 'feature/auto-step-triggers'
into main"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   bbece0a..2ed5baa  main -> main
```

5.10 Step 9: Bonus 3 – Custom Hotkey Re-binding & Step Description Editor

- **Goal**: Allow users to configure their own hotkeys in Settings, review/edit step descriptions during or after sessions, and remove emoji stickers for a clean interface.

```
$ git checkout -b feature/hotkey-config-and-step-editor
$ git add .
$ git commit -m "feat(ui): add custom hotkey configuration, step description editor, and clean
sticker-free UI"
$ git push -u origin feature/hotkey-config-and-step-editor
$ git checkout main
$ git merge --no-ff feature/hotkey-config-and-step-editor -m "Merge branch 'feature/hotkey-
config-and-step-editor' into main"
$ git push origin main
To https://github.com/hachemite/recordia-ilia.git
   2ed5baa..d94d961  main -> main
```

6 Quality Assurance & Test Verification

A robust automated test suite was maintained throughout the development lifecycle to ensure continuous regression protection across all modules.

```
$ python -m unittest discover -s tests -v
test_init_and_state (test_auto_stepper.TestAutoStepper) ... ok
test_periodic_timer_trigger (test_auto_stepper.TestAutoStepper) ... ok
test_stop_when_idle (test_auto_stepper.TestAutoStepper) ... ok
test_directory_creation (test_file_manager.TestFileManager) ... ok
test_generate_recording_path (test_file_manager.TestFileManager) ... ok
test_generate_screenshot_path (test_file_manager.TestFileManager) ... ok
test_update_base_directory (test_file_manager.TestFileManager) ... ok
test_init_and_bindings (test_hotkeys.TestHotkeyManager) ... ok
test_register_hotkey (test_hotkeys.TestHotkeyManager) ... ok
test_stop_when_not_started (test_hotkeys.TestHotkeyManager) ... ok
test_start_and_stop_recording (test_recorder.TestScreenRecorder) ... ok
```

```
test_capture_fullscreen (test_screenshot.TestScreenshotEngine) ... ok
test_capture_region (test_screenshot.TestScreenshotEngine) ... ok
test_finish_session_generates_html_and_markdown_reports
(test_session_manager.TestSessionManager) ... ok
test_record_steps_with_metadata (test_session_manager.TestSessionManager) ... ok
test_start_session_creates_directory_structure (test_session_manager.TestSessionManager) ... ok
test_update_step_description (test_session_manager.TestSessionManager) ... ok
test_default_values (test_settings.TestSettings) ... ok
test_set_and_save_persistence (test_settings.TestSettings) ... ok
test_get_active_window_info_structure (test_window_info.TestWindowInfo) ... ok
----------------------------------------------------------------------
Ran 20 tests in 2.579s
OK
```

7 How to Run & Verify

1. **Clone the repository**:

```
$ git clone https://github.com/hachemite/recordia-ilia.git
$ cd recordia-ilia
```

2. **Install dependencies**:

```
$ pip install -r requirements.txt
```

3. **Execute the automated test suite**:

```
$ python -m unittest discover -s tests -v
```

4. **Launch the desktop application**:

```
$ python src/main.py
```

8 Conclusion & Collaborative Reflections

The development of RecordiaILIA served as an exemplary practical demonstration of professional version control and agile collaboration:

1. **Conflict Avoidance**: By strictly separating features into dedicated branches and pulling upstream before merging, zero merge conflicts occurred throughout the project.
2. **Transparent Audit Trail**: Non-fast-forward (`--no-ff`) merges ensure that even years after release, any team member or evaluator can visually trace the origin, testing, and integration of each individual feature in the Git commit graph.
3. **Automated Governance**: Incorporating issue closure triggers (`Fixes #X`) directly into version control messages harmonized the Git repository with GitHub's project management board, eliminating manual synchronization overhead.
4. **Software Excellence**: Beyond Git mechanics, the resulting application is a complete, feature-rich desktop utility providing automated problem steps recording, customizable hotkeys, and multi-threaded screen recording.